

Exercise PROG.20 – Fun with Arrays

2022-03-30

Instructions

- Copy the starter project in a separate directory on your network share
- For each task create a `<TaskName>.cs` file within that project if not present already
- Solve the following tasks
- Make sure to run `CleanSolutionDir.ps1` before you pack and hand in your solution

1 Sorting

After Bubble- and Selection-Sort you are now tasked to sort an array using the Insertion-Sort algorithm. To this end implement the method `SortInPlace` which – as the name indicates – performs the sorting in-place.

2 Sudoku

You are going to implement the game Sudoku. However, to keep the task simple, we are going to define the following limitations:

- We only process 9x9 sized sudokus
 - Still, make sure that your methods are as general purpose as possible!
- It is not necessary to generate sudoku exercises – we will only validate if the user input is valid and leads to a completed sudoku
 - A sample sudoku is provided with the starter code and can be used.

In fact, the main algorithmic challenge of this exercise is to split up the 2D array which represents the sudoku into rows, columns and segments – see fig. 1. Then individually check those if they are invalid, valid or complete. Additionally, user input logic and sudoku playing field output is required.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Figure 1: Sudoku Areas

Valid

Both a sequence, see tab. 1, and a segment, see tab. 2, are *valid*, if:

- Does not contain *duplicate* numbers
- Only contains numbers which are *in range*
 - In our case that would be $[1,9]$
 - With the exception of 0 which indicates that a cell is *empty*

Invalid	3	0	5	5	8	0	2	4	0
Valid	3	0	0	5	8	0	2	4	0

Table 1: Valid & Invalid Sequence

Invalid			Valid		
3	0	5	3	0	0
5	8	0	5	8	0
2	4	0	2	4	0

Table 2: Valid & Invalid Segment

Complete

Both a sequence, see tab. 3, and a segment, see tab. 4, are *complete*, if:

- Is valid¹
- Contains *all* numbers which are *in range*
 - In our case that would be $[1,9]$
 - The order is *not* important
 - 0 can *not* occur

Incomplete	3	7	6	5	8	0	2	4	1
Complete	3	7	6	5	8	9	2	4	1

Table 3: Complete & Incomplete Sequence

Incomplete			Complete		
3	7	6	3	7	6
5	8	0	5	8	9
2	4	1	2	4	1

Table 4: Complete & Incomplete Segment



If all sequences are valid/complete the segments should be implicitly as well. But for the sake of training how to work with arrays we will check both anyway.

¹Hint: a complete area is implicitly valid as it cannot contain invalid numbers

Methods

The purpose of each method is defined by XML-Doc in the starter code and can be verified using the provided unit tests. Thus, no detailed explanation of methods, parameters or return values is provided here this time – with the exception of the console output, see below. Get used to working with code comments! Also, be careful to *reuse* methods whenever possible to avoid code duplication.

Game Run

The game repeats the following steps until either the sudoku is complete or the user has given up.

1. Print the sudoku field
2. Ask the user for:
 - A valid row
 - A valid column
 - A valid number
3. Try if the number can be inserted at the given position
 - This is *not* possible if the result would be an *invalid* sequence or segment
 - If the user entered such a number ask if they want to continue or abort $\Rightarrow$ if they want to abort, end the game; if not go back to the first step
4. Insert the number in the sudoku field
5. Check if with that last number added the sudoku is complete
 - If it is, end the game with some congratulations to the user
 - If not, go back to the first step

The following screenshots show:

- Entering a valid number, see fig. 2
- Entering a valid number which is invalid at the specified position, see fig. 3
- Aborting the game, see fig. 4
- Winning the game, see fig. 5

If you now want to get into the 'Sudoku mood' you can play a couple of free rounds here.

```

Fun with Arrays Exercise

*** Sudoku ***
+++++
+ ? + ? + 1 + 2 + ? + ? + 8 + 7 + ? +
+ ? + ? + 6 + ? + 8 + ? + ? + 2 + 4 +
+ ? + 8 + ? + ? + 7 + 3 + ? + ? + 5 +
+ 6 + 2 + ? + 1 + 3 + ? + ? + 8 + ? +
+ 8 + ? + ? + 9 + 4 + ? + ? + 5 + 2 +
+ 5 + 9 + 4 + ? + ? + 8 + 3 + ? + 6 +
+ 3 + ? + 9 + ? + ? + ? + 5 + 4 + ? +
+ 1 + ? + ? + ? + 9 + ? + 2 + ? + 8 +
+ ? + ? + ? + ? + 5 + 7 + ? + ? + ? +
Enter selected row in range [1,9]: -3
Invalid input, try again...
Enter selected row in range [1,9]: 0
Invalid input, try again...
Enter selected row in range [1,9]: 10
Invalid input, try again...
Enter selected row in range [1,9]: 1
Enter selected col in range [1,9]: 80
Invalid input, try again...
Enter selected col in range [1,9]: 1
Enter number to enter in range [1,9]: 0
Invalid input, try again...
Enter number to enter in range [1,9]: 45
Invalid input, try again...
Enter number to enter in range [1,9]: -1
Invalid input, try again...
Enter number to enter in range [1,9]: 4
+ 4 + ? + 1 + 2 + ? + ? + 8 + 7 + ? +
+ ? + ? + 6 + ? + 8 + ? + ? + 2 + 4 +
+ ? + 8 + ? + ? + 7 + 3 + ? + ? + 5 +
+ 6 + 2 + ? + 1 + 3 + ? + ? + 8 + ? +
+ 8 + ? + ? + 9 + 4 + ? + ? + 5 + 2 +
+ 5 + 9 + 4 + ? + ? + 8 + 3 + ? + 6 +
+ 3 + ? + 9 + ? + ? + ? + 5 + 4 + ? +
+ 1 + ? + ? + ? + 9 + ? + 2 + ? + 8 +
+ ? + ? + ? + ? + 5 + 7 + ? + ? + ? +
Enter selected row in range [1,9]:

```

Figure 2: Entering a valid number

```

+++++
+ ? ? 1 2 ? ? 8 7 ? +
+++++
+ ? ? 6 ? 8 ? ? 2 4 +
+++++
+ ? 8 ? ? 7 3 ? ? 5 +
+++++
+ 6 2 ? 1 3 ? ? 8 ? +
+++++
+ 8 ? ? 9 4 ? ? 5 2 +
+++++
+ 5 9 4 ? ? 8 3 ? 6 +
+++++
+ 3 ? 9 ? ? ? 5 4 ? +
+++++
+ 1 ? ? ? 9 ? 2 ? 8 +
+++++
+ ? ? ? ? 5 7 ? ? ? +
+++++
Enter selected row in range [1,9]: 1
Enter selected col in range [1,9]: 2
Enter number to enter in range [1,9]: 2
The number you entered is not valid (at this position)!
Do you want to continue the game (y/n)? y
+++++
+ ? ? 1 2 ? ? 8 7 ? +
+++++
+ ? ? 6 ? 8 ? ? 2 4 +
+++++
+ ? 8 ? ? 7 3 ? ? 5 +
+++++
+ 6 2 ? 1 3 ? ? 8 ? +
+++++
+ 8 ? ? 9 4 ? ? 5 2 +
+++++
+ 5 9 4 ? ? 8 3 ? 6 +
+++++
+ 3 ? 9 ? ? ? 5 4 ? +
+++++
+ 1 ? ? ? 9 ? 2 ? 8 +
+++++
+ ? ? ? ? 5 7 ? ? ? +
+++++
Enter selected row in range [1,9]:

```

Figure 3: Entering a valid number in wrong position

```

+++++
+ 4 ? 1 2 ? ? 8 7 ? +
+++++
+ ? ? 6 ? 8 ? ? 2 4 +
+++++
+ ? 8 ? ? 7 3 ? ? 5 +
+++++
+ 6 2 ? 1 3 ? ? 8 ? +
+++++
+ 8 ? ? 9 4 ? ? 5 2 +
+++++
+ 5 9 4 ? ? 8 3 ? 6 +
+++++
+ 3 ? 9 ? ? ? 5 4 ? +
+++++
+ 1 ? ? ? 9 ? 2 ? 8 +
+++++
+ ? ? ? ? 5 7 ? ? ? +
+++++
Enter selected row in range [1,9]: 9
Enter selected col in range [1,9]: 4
Enter number to enter in range [1,9]: 5
The number you entered is not valid (at this position)!
Do you want to continue the game (y/n)? n
Better luck next time!

```

Figure 4: Aborting the game

```

+++++
+ 5 + 3 + 4 + 6 + 7 + 8 + 9 + 1 + 2 +
+++++
+ 6 + 7 + 2 + 1 + 9 + 5 + 3 + 4 + 8 +
+++++
+ 1 + 9 + 8 + 3 + 4 + 2 + 5 + 6 + 7 +
+++++
+ 8 + 5 + 9 + 7 + 6 + 1 + 4 + 2 + 3 +
+++++
+ 4 + 2 + 6 + 8 + ? + 3 + 7 + 9 + 1 +
+++++
+ 7 + 1 + 3 + 9 + 2 + 4 + 8 + 5 + 6 +
+++++
+ 9 + 6 + 1 + 5 + 3 + 7 + 2 + 8 + 4 +
+++++
+ 2 + 8 + 7 + 4 + 1 + 9 + 6 + 3 + 5 +
+++++
+ 3 + 4 + 5 + 2 + 8 + 6 + 1 + 7 + 9 +
+++++
Enter selected row in range [1,9]: 5
Enter selected col in range [1,9]: 5
Enter number to enter in range [1,9]: 5
Congratulations, you did it!
+++++
+ 5 + 3 + 4 + 6 + 7 + 8 + 9 + 1 + 2 +
+++++
+ 6 + 7 + 2 + 1 + 9 + 5 + 3 + 4 + 8 +
+++++
+ 1 + 9 + 8 + 3 + 4 + 2 + 5 + 6 + 7 +
+++++
+ 8 + 5 + 9 + 7 + 6 + 1 + 4 + 2 + 3 +
+++++
+ 4 + 2 + 6 + 8 + 5 + 3 + 7 + 9 + 1 +
+++++
+ 7 + 1 + 3 + 9 + 2 + 4 + 8 + 5 + 6 +
+++++
+ 9 + 6 + 1 + 5 + 3 + 7 + 2 + 8 + 4 +
+++++
+ 2 + 8 + 7 + 4 + 1 + 9 + 6 + 3 + 5 +
+++++
+ 3 + 4 + 5 + 2 + 8 + 6 + 1 + 7 + 9 +
+++++

```

Figure 5: Winning the game